EN3563 Robotics
Laboratory Experiment 01

Title: Spatial Descriptions and Orientation Representations using Robotics Toolbox

1. Introduction

The relationship between two rigid bodies can be established by first attaching a coordinate frame to each rigid body and describing the relative position and relative orientation between the two coordinate frames (Fig. 1). To describe relative orientation, we use a rotation matrix.

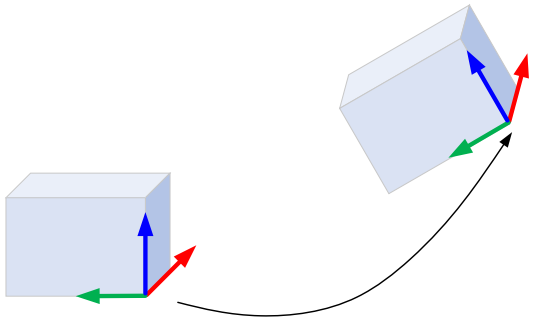


Figure 1: Establishing a relationship between two rigid bodies

The Robotics Toolbox is a MATLAB toolbox software that supports research and teaching into arm-type and mobile robotics. It contains functions and classes to represent orientation and pose in 2D and 3D (SO(2), SE(2), SO(3), SE(3)) as matrices, quaternions, twists, triple angles, and matrix exponentials. You are expected to have correctly configured the Robotics Toolbox to proceed with the remainder of this laboratory experiment.

From this experiment you will learn the following:

- Usage of MATLAB Robotics Toolbox
- Reinforce the understanding of spatial descriptions and orientation representations.

2. Theory

This section describes the underlying theories associated with spatial descriptions and orientation representations.

2.1. Rotation Matrix

A rotation matrix is an $n \times n$ matrix that belongs to the special orthogonal group $SO(n)$ of order n. It can be used to represent relative orientation between two coordinate frames.

2D Rotation

Figure 2 shows two coordinate frames, with frame $o_1x_1y_1$ oriented at an angle θ with respect to frame $o_0x_0y_0$. The resulting rotation matrix is given by,

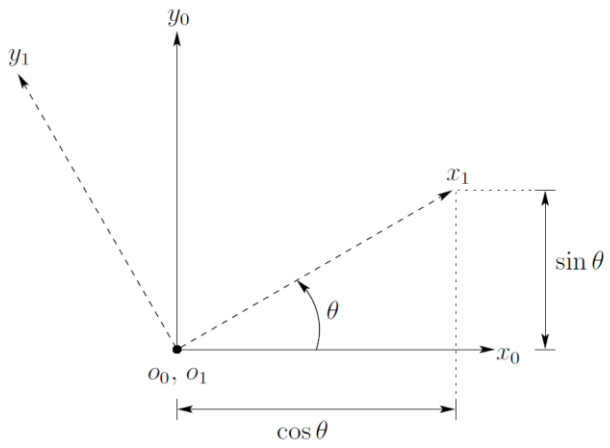


Figure 2: Coordinate frame $o_1x_1y_1$ is oriented at an angle θ with respect to $o_0x_0y_0$

$$R_1^0 = \begin{bmatrix} x_1 \cdot x_0 & y_1 \cdot x_0 \\ x_1 \cdot y_0 & y_1 \cdot x_0 \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}. \tag{1}$$

3D Rotation

To obtain the rotation matrix for three dimensions, each axis of the frame $o_1x_1y_1$ is projected onto frame $o_0x_0y_0$. The resulting rotation matrix is given by,

$$R_1^0 = \begin{bmatrix} x_1 \cdot x_0 & y_1 \cdot x_0 & z_1 \cdot x_0 \\ x_1 \cdot y_0 & y_1 \cdot y_0 & z_1 \cdot y_0 \\ x_1 \cdot z_0 & y_1 \cdot z_0 & z_1 \cdot z_0 \end{bmatrix}. \tag{2}$$

For each axis, the basic rotation matrices are given by,

$$R_{z,\theta} = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \tag{3}$$

$$R_{x,\theta} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\theta & -\sin\theta \\ 0 & \sin\theta & \cos\theta \end{bmatrix} \tag{4}$$

$$R_{y,\theta} = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix} \tag{5}$$

2.2. Rotational Transformations in Two Frames with Common Origin

Rotation matrix R_1^0 can be used to transform the coordinates of a point from one frame to another.

$$p^0 = R_1^0 p^1 \tag{6}$$

2.3. Rotation Matrix as an Operator

Rotation matrix R can be operated on a vector (e.g. position vector) to rotate it in the same coordinate frame.

$$q^0 = R p^0 \tag{7}$$

2.4. Parameterization of Rotations

The nine elements in a general SO(3) rotation matrix R are not independent. An arbitrary rotation can be represented using three independent quantities.

Euler Representation

In Z-Y-X Euler angles, a general rotation is given by the outcome of following sequence:

Rotate about current Z axis by ϕ angle Rotate about current Y axis by θ angle Rotate about current X axis by ψ angle

Fixed Angle Representation

In fixed angles, a general rotation is given by the outcome of following sequence:

Rotate about fixed X axis by ψ angle Rotate about fixed Y axis by θ angle Rotate about fixed Z axis by ϕ angle

Roll-Pitch-Yaw

There can be some confusion around the terms *roll-pitch-yaw angles* due to various definitions on different textbooks or toolboxes. Carefully go through the documentation and conduct simple experiments to figure out the sequence of rotations as well as the axes used.

3. Procedure

Follow the subsequent procedure using notation as described in section 5.

Spatial Descriptions

3.1. Visualize the default 2D coordinate frame $\{0\}$ in a MATLAB figure. Limit the plot area to X: $-4, 7$ and Y:$-2, 7$ and enable grid.

3.2. Consider a 2D point p with position vector $[5\ \ 6]^T$ in frame $\{0\}$. Visualize p using a blue arrow.

3.3. Rotate the default coordinate frame $\{0\}$ counterclockwise by 45^0 using animation techniques. Visualize the new coordinate frame $\{1\}$ in red color. Find p in frame $\{1\}$.

3.4. Consider another 2D point q with position vector $[-3\ \ 2]^T$ in frame $\{1\}$. Visualize q using a red arrow.

3.5. Apply a 68^0 counterclockwise rotation to the position vector of p to obtain the new 2D point r. Visualize r using a green arrow.

Orientation Representations

3.6. In a new MATLAB figure, visualize the default 3D coordinate frame $\{0\}$. Limit the plot area to $-1, 2$ for all X, Y, and Z directions.

3.7. Another 3D coordinate frame $\{1\}$ is obtained as follows:

> Rotate the default coordinate frame about X axis for $+15^0$.
> Rotate the current coordinate frame about the new Y axis for $+25^0$.
> Rotate the current coordinate frame about the new Z axis for $+35^0$.

Using suitable rotation functions, obtain 3x3 rotation matrices for each rotation, and the final rotation R_1^0.
Using animation techniques, visualize successive rotations that leads to R_1^0. Ultimately, visualize frame $\{1\}$ using red color.

3.8. Find the default roll-pitch-yaw angle definition for the toolbox. Note, however, that it is customizable using parameters.

3.9. For the following 3x3 rotation matrix (same matrix from the lecture), use the toolbox to find ψ (roll about X axis), θ (pitch about Y axis) and φ (yaw about Z axis) angles in degrees. Confirm your answer by doing the opposite conversion, and also using the product of basic rotation matrices.

$$R = \begin{bmatrix} 0.8138 & 0.0400 & 0.5798 \\ 0.2962 & 0.8298 & -0.4730 \\ -0.5000 & 0.5567 & 0.6634 \end{bmatrix}$$

4. MATLAB Robotics Toolbox Reference

4.1. Position Vector

`plot_arrow`
Draw an arrow in 2D or 3D

4.2. Rotation Matrix

`rot2`: SO(2) rotation matrix
`rot3`: SO(3) rotation matrix

4.3. Coordinate Frame

`trplot2`: 2D Coordinate frame
`trplot`: 3D Coordinate frame

Related commands
`hold on`: multiple frames can be added
`axis`: limit the plot area
`grid`: grid can be added to the plot

4.4. Animations

`tranimate2`: Animate a 2D coordinate frame
`tranimate`: Animate a 3D coordinate frame

↳ options:
`cleanup`: remove the frame at end of animation

4.5. Rotations

`rotx`: SO(3) rotation about X axis
`roty`: SO(3) rotation about Y axis
`rotz`: SO(3) rotation about Z axis

4.6. Conversions

`rpy2r`: roll-pitch-yaw angles to SO(3) rotation matrix
`tr2rpy`: convert SO(3) or SO(3) matrix to roll-pitch-yaw angles

5. Notation

Use a meaningful notation in MATLAB similar to the following:

Notation	Meaning
p_in_a	Point **p** represented in frame {a}
R_a_in_b	Orientation of frame {a} with respect to frame {b}
R_x_30	30^0 rotation about X axis

Answer Sheet

Index No: 220350T

1. MATLAB code for 3.1 ~ 3.5.

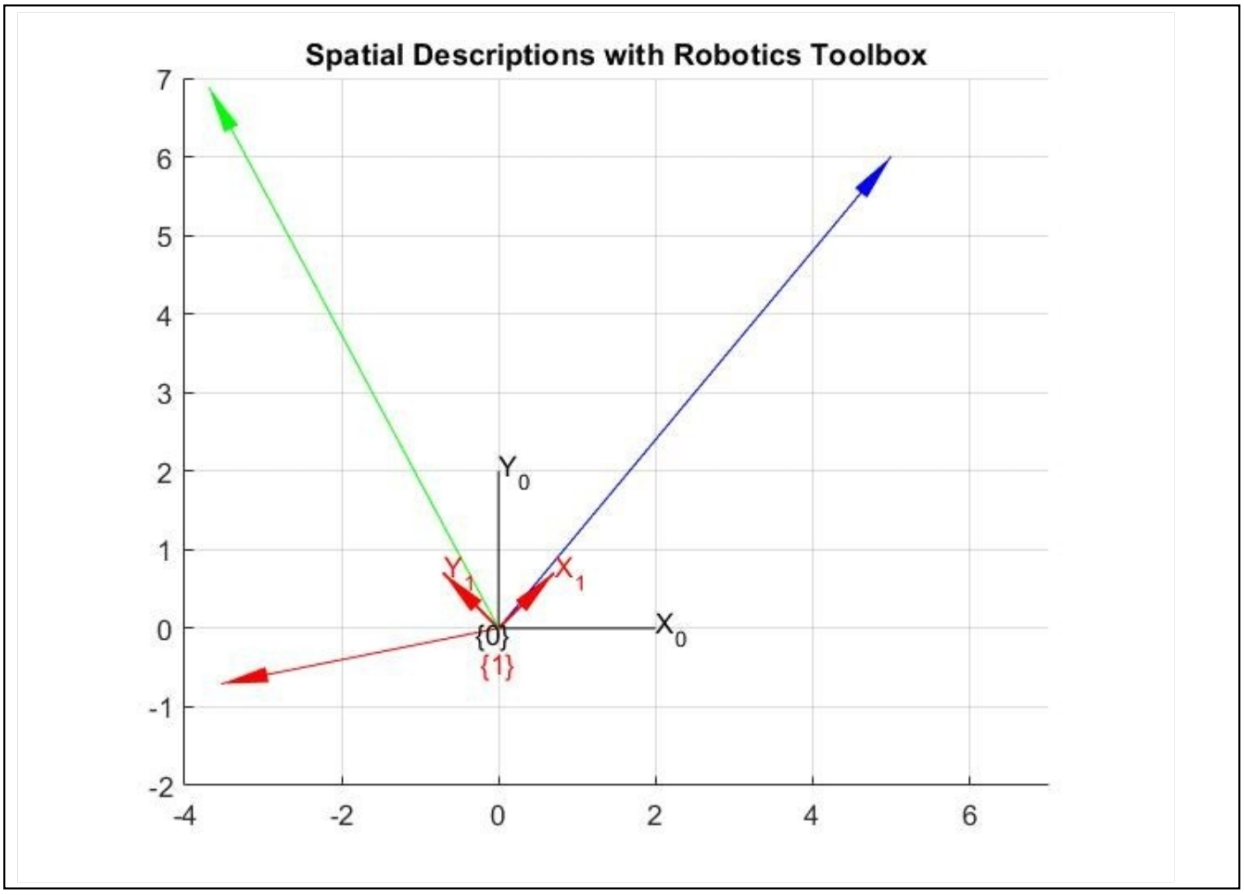
```
startup_rvc.m | robotic_Lab1.m | roboticslab12.m

 7    % 3.1 Default frame {0}
 8 -  trplot2(SE2(), 'frame', '0', 'color', 'k', 'length', 2);
 9 -  axis([-4,7,-2,7])
10 -  grid on;
11    % 3.2 Point p = [5;6] in frame {0}
12 -  p_in_0 = [5; 6];
13 -  plot_arrow([0 0], p_in_0', 'b');
14
15    % 3.3 Frame {1} rotated 45° CCW from {0}
16 -  theta = deg2rad(45);
17 -  R1_in_0 = rot2(theta);
18 -  tranimate2(R1_in_0, 'frame', '1', 'color', 'r', 'arrow')
19
20    % Transform point p into frame {1}
21 -  p_in_1 = R1_in_0' * p_in_0;  % transformation
22 -  disp('p in frame {1}:');
23 -  disp(p_in_1);
24
25    % 3.4 Define q = [-3;2] in frame {1}
26 -  q_in_1 = [-3; 2];
27 -  q_in_0 = R1_in_0 * q_in_1;   % expressed in frame {0} for visualizatio
28 -  plot_arrow([0 0], q_in_0', 'r');
29
30    % 3.5 Rotate p by 68° CCW in {0}
31 -  phi = deg2rad(68);
32 -  Rphi = rot2(phi);
33 -  r_in_0 = Rphi * p_in_0;
34 -  plot_arrow([0 0], r_in_0', 'g');
35
```

```
p in frame {1}:
    7.7782
    0.7071

fx >>
```

2. Final output MATLAB figure for the operations in 3.1 ~ 3.5.



3. p^1 for 3.3:

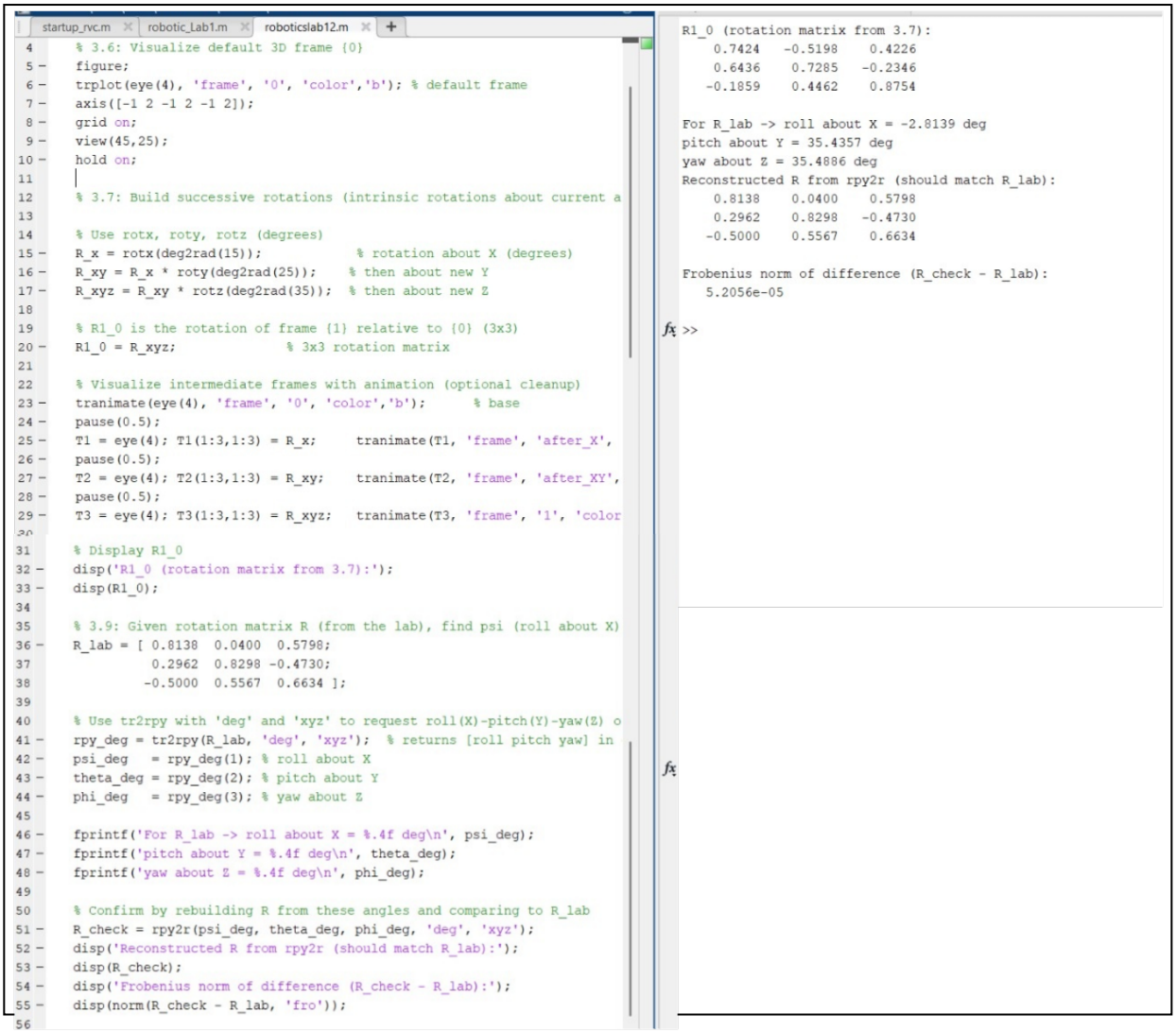
```
p in frame {1}:
    7.7782
    0.7071
```

$p' = [7.7782, 0.7071]$

4. R_1^0 for 3.7.

```
R1_0 (rotation matrix from 3.7):
    0.7424   -0.5198    0.4226
    0.6436    0.7285   -0.2346
   -0.1859    0.4462    0.8754
```

5. MATLAB code for 3.6 ~ 3.9.



```
% 3.6: Visualize default 3D frame {0}
figure;
trplot(eye(4), 'frame', '0', 'color','b'); % default frame
axis([-1 2 -1 2 -1 2]);
grid on;
view(45,25);
hold on;

% 3.7: Build successive rotations (intrinsic rotations about current a

% Use rotx, roty, rotz (degrees)
R_x = rotx(deg2rad(15));            % rotation about X (degrees)
R_xy = R_x * roty(deg2rad(25));    % then about new Y
R_xyz = R_xy * rotz(deg2rad(35));  % then about new Z

% R1_0 is the rotation of frame {1} relative to {0} (3x3)
R1_0 = R_xyz;              % 3x3 rotation matrix

% Visualize intermediate frames with animation (optional cleanup)
tranimate(eye(4), 'frame', '0', 'color','b');      % base
pause(0.5);
T1 = eye(4); T1(1:3,1:3) = R_x;     tranimate(T1, 'frame', 'after_X',
pause(0.5);
T2 = eye(4); T2(1:3,1:3) = R_xy;    tranimate(T2, 'frame', 'after_XY',
pause(0.5);
T3 = eye(4); T3(1:3,1:3) = R_xyz;   tranimate(T3, 'frame', '1', 'color

% Display R1_0
disp('R1_0 (rotation matrix from 3.7):');
disp(R1_0);

% 3.9: Given rotation matrix R (from the lab), find psi (roll about X)
R_lab = [ 0.8138  0.0400  0.5798;
          0.2962  0.8298 -0.4730;
         -0.5000  0.5567  0.6634 ];

% Use tr2rpy with 'deg' and 'xyz' to request roll(X)-pitch(Y)-yaw(Z) o
rpy_deg = tr2rpy(R_lab, 'deg', 'xyz');  % returns [roll pitch yaw] in
psi_deg   = rpy_deg(1); % roll about X
theta_deg = rpy_deg(2); % pitch about Y
phi_deg   = rpy_deg(3); % yaw about Z

fprintf('For R_lab -> roll about X = %.4f deg\n', psi_deg);
fprintf('pitch about Y = %.4f deg\n', theta_deg);
fprintf('yaw about Z = %.4f deg\n', phi_deg);

% Confirm by rebuilding R from these angles and comparing to R_lab
R_check = rpy2r(psi_deg, theta_deg, phi_deg, 'deg', 'xyz');
disp('Reconstructed R from rpy2r (should match R_lab):');
disp(R_check);
disp('Frobenius norm of difference (R_check - R_lab):');
disp(norm(R_check - R_lab, 'fro'));
```

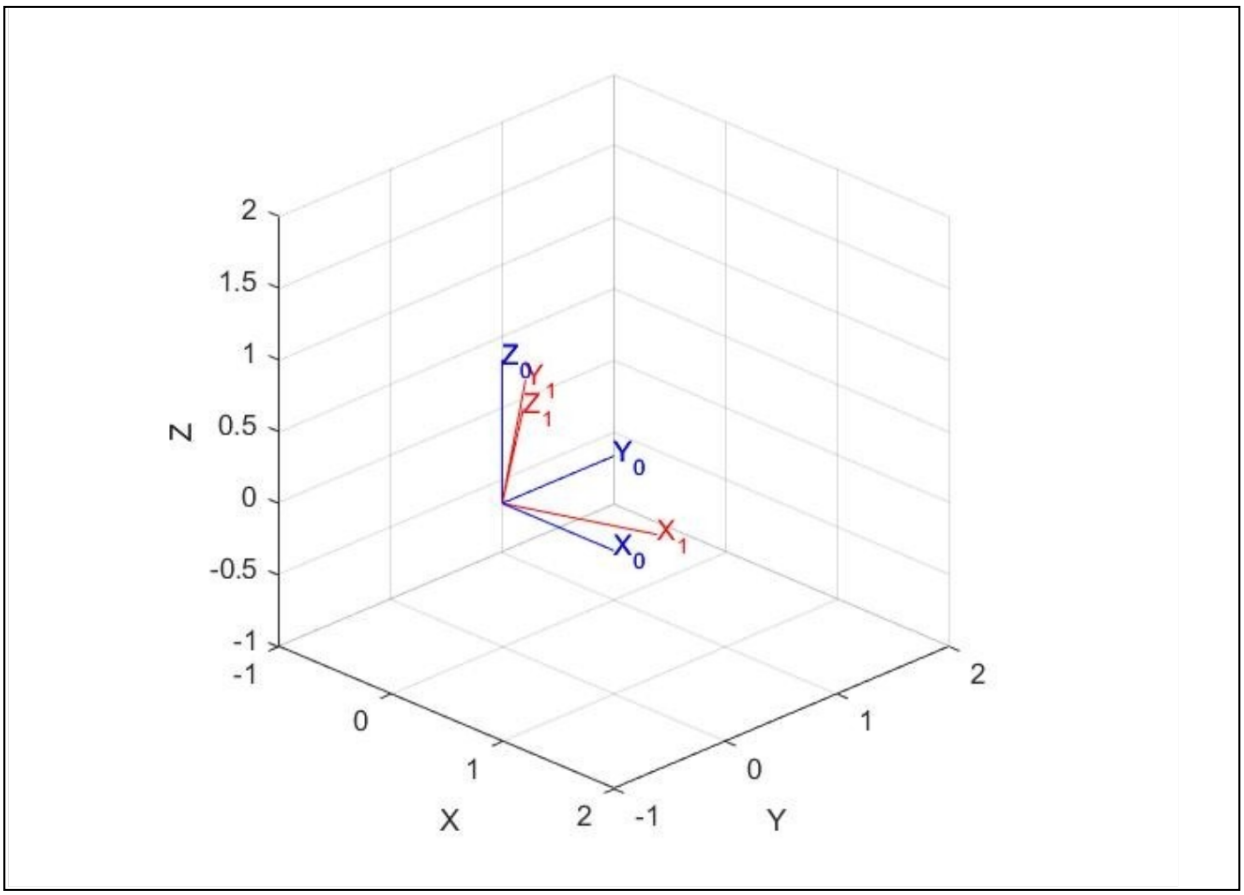
```
R1_0 (rotation matrix from 3.7):
    0.7424   -0.5198    0.4226
    0.6436    0.7285   -0.2346
   -0.1859    0.4462    0.8754

For R_lab -> roll about X = -2.8139 deg
pitch about Y = 35.4357 deg
yaw about Z = 35.4886 deg
Reconstructed R from rpy2r (should match R_lab):
    0.8138    0.0400    0.5798
    0.2962    0.8298   -0.4730
   -0.5000    0.5567    0.6634

Frobenius norm of difference (R_check - R_lab):
   5.2056e-05

>>
```

6. Final output MATLAB figure for the operations in 3.6 ~ 3.9.



7. Default roll-pitch-yaw angle definition for the toolbox.

Default Roll-Pitch-Yaw order → ZYX (yaw, pitch, roll)

8. For 3.9,
ψ: -2.8139° θ: 35.4357° φ: 35.4886°